

TOPICS

Query, Subquery and Joins

- Group By and Having
- Query and Subquery, Use of IN ,ANY, ALL
- Creation and manipulation of Object – Indexes, Views, Sequences, Synonym
- Joining tables, types of joins (equi join, outer join, self join)

→ **GROUP BY** :- The GROUP BY clause is another section of the SELECT statement. This optional clause tells Oracle to group rows based on distinct values that exist for specified columns. The GROUP BY clause creates a data set, containing several sets of records grouped together based on a condition.

- **Syntax** :

```
SELECT col1, col2, colN, group_function(Column)
FROM table name WHERE condition
GROUP BY col1, col2, colN
ORDER BY col
```

GROUP BY and HAVING clauses are parallel to the order by and where clause, except that they act on record set, and not in individual records.

In GROUP BY clause specifies columns whose values determine the basis for grouping rows. When using the GROUP BY clause, make sure that all columns in the SELECT list that are not group functions are included in the GROUP BY clause.

- **Guidelines** :

- If you include a group function in a SELECT clause, you cannot select individual results as well, unless the individual column appears in the group by clause. You receive an error message if you fail to include the column list in the GROUP BY clause.
- Using a WHERE clause, you can exclude rows before dividing them into groups.
- You must include the columns in the GROUP BY clause.
- You cannot use a column alias in the GROUP BY clause.
- By default, rows are sorted by ascending order of the column included in the GROUP BY list. You can override this by using the ORDER BY clause.

- **Example** :

```
SELECT deptid, AVG(sal) FROM emp GROUP BY deptid;
```

The GROUP BY column does not have to be in the SELECT clause.

```
SELECT AVG(sal) from Emp GROUP BY deptid ;
```

- **GROUP BY more than one column** :

Sometimes you need to see results for groups within groups.

Emp

Deptid

Desig

Sal

The EMP table is grouped first by department number and, within that grouping, by designation . F.e. The Four clerks in department 10 are grouped together and a single result is produced for all clerks within the group.

```
SELECT deptid, desig, sum(sal) FROM emp GROUP BY deptid, design;
```

- **Illegal queries using GROUP functions :**
SELECT deptid, count(desig) FROM emp;

Whenever you use a mixture of individual items (DEPTID) and group function (COUNT) in the same SELECT statement, you must include a GROUP BY clause that specifies the individual items in this case (DEPTID). If the GROUP BY is missing, Then the error message “ORA-00937: not a single-group group function” appears. You can correct the error by adding GROUP BY clause.

SELECT deptid, count(desig) FROM emp GROUP BY deptid;

- **Nesting Group Functions :** Group function can be nested to a depth of two.
SELECT Max(Avg(sal)) FROM emp GROUP BY deptid;

→ **HAVING clause :** The HAVING clause can be used in conjunction with the GROUP BY clause. HAVING imposes a condition on the GROUP BY clause, which further filters the groups created by the GROUP BY clause. Each column specification specified in the HAVING clause must occur within a statistical function or must occur in the list of column named in the GROUP BY clause. In the same way that you use the WHERE clause to restrict the rows that you select, you use the HAVING clause to restrict groups. You use the HAVING clause to specify which groups are to be displayed.

SELECT column, group function
FROM table
WHERE condition
GROUP BY group_by_expression
HAVING group_condition
ORDER BY column;

SELECT deptid, avg(sal) FROM emp WHERE avg(sal) > 8000 GROUP BY deptid;

The WHERE clause cannot be used to restrict groups. The SELECT statement above results in an error because it uses the WHERE clause to restrict the display of average salaries of those departments that have an average salary greater than 8000. You can correct the error by using the HAVING clause to restrict the groups.

SELECT deptid, avg(sal) FROM emp HAVING avg(sal) > 8000 GROUP BY deptid;

Group_Condition :- Restricts the groups of rows returned to those groups for which the specified condition is true.

The Oracle server performs the following steps when you use HAVING clause :

- 1) Rows are grouped
- 2) The group function is applied to the group
- 3) The groups that match the criteria in the HAVING clause are displayed.

The HAVING clause can precede the GROUP BY clause, but it is recommended that you place the GROUP BY clause first because that is more logical. Groups are formed and group functions are calculated before the HAVING clause is applied to the groups in the SELECT list.

→ **SUBQUERIES** : Suppose you want to write a query to find out who earns a salary greater than Scott's salary. To solve this problem, you need two queries : One to find what Scott earns, and a second query to find who earns more than that amount. You can solve this problem by combining the two queries, placing one query inside the other query.

A Subquery is a form of an SQL statement that appears inside another SQL statement. It is also termed as nested query. The statement containing a subquery is called a Parent statement or Outer query. The parent statement uses the rows by the subquery or inner query or child query.

It can be used for following :

- 1) To insert records in a target table.
- 2) To create tables and insert records in the table created.
- 3) To update records in a target table.
- 4) To create views
- 5) To provide values for conditions in WHERE, HAVING, IN and so on used with SELECT, UPDATE, and DELETE statements.

• **Syntax** :-

SELECT select_list FROM table

WHERE expr operator (SELECT select list FROM table);

Subquery can be very useful when you need to select rows from a table with a condition that depends on the data in the table itself. You can place the subquery in three clauses of SQL like WHERE, HAVING, FROM clauses.

In syntax Operator :- include a comparison condition such as single row operators (>, =, >=, <=, <, <>) and multiple row operators (IN, ANY, ALL).

• **Example** :-

SELECT name FROM emp

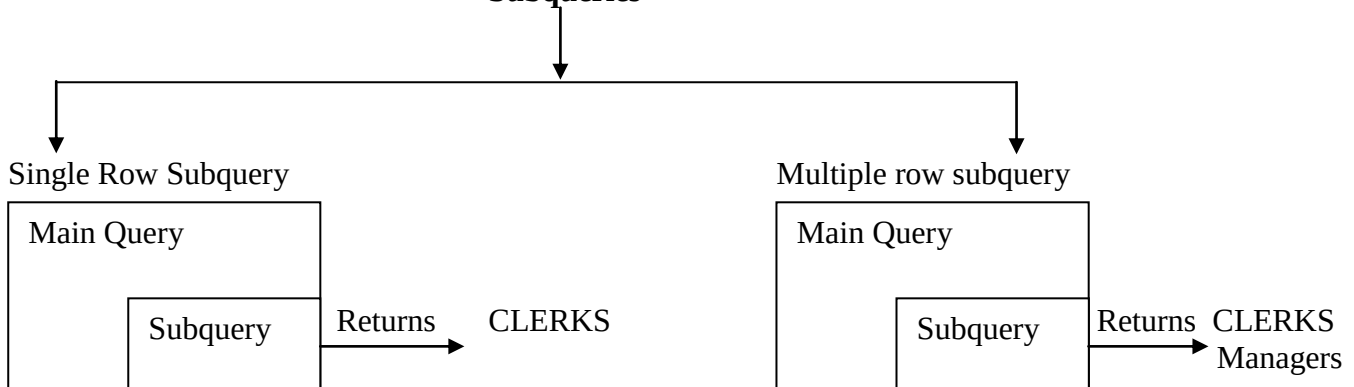
WHERE sal > (SELECT sal FROM emp WHERE name = 'SCOTT');

• **Guideline** :-

- 1) A subquery must be enclosed in parantheses.
- 2) Place the subquery on the right side of the comparison condition for readability
- 3) Starting with release oracle 8i, an ORDER BY clause can be used and is required in the subquery to perform Top – N analysis.
- 4) Two classes of comparison condition are used in subqueries : Single row operators and multiple row operators.

• **Types of SUBQUERIES** :-

Subqueries



- Single Row Subqueries :- Queries that return only one row from the inner SELECT statement.
- Multiple row Subqueries :- Queries that return more than one row from inner SELECT statement

There are also multiple column subqueries:- Queries that return more than one column from the inner SELECT statement.

- Single Row Subqueries :- A single row subquery is one that returns one row from the inner SELECT statement. This type of subquery uses a single row operator like (>, =, >=, <=, <, <>)

SELECT name, desg FROM emp WHERE desg = (SELECT design FROM emp WHERE empno=7366);

SELECT name, design, sal FROM emp
WHERE desg = (SELECT design FROM emp WHERE empno=7366) AND
Sal > (SELECT sal FROM emp WHERE empno= 7843);

SELECT name, design, sal FROM emp WHERE sal = (SELECT min(sal) FROM emp);

- The HAVING clause with subqueries :

SELECT deptid, min(sal) FROM emp
GROUP BY deptid HAVING min(sal) > (Select min(sal) FROM employees
where deptid = 20);

- Find the job with the lowest average salary

SELECT design, avg(sal) FROM emp GROUP BY desg
HAVING avg(sal) = (SELECT min(avg(sal)) FROM emp GROUP BY desg);

- Using Sub-query in the FROM clause :

A sub-query can be used in the FROM clause of the SELECT statement. The concept of using a sub-query in the FROM clause of the SELECT statement is called an inline view. A sub-query in the FROM clause of the SELECT statement defines a data source from that particular SELECT statement.

SELECT a.name, a.sal, a.deptno, b.maxsal FROM emp a, (SELECT deptid, max(sal)
maxsal FROM emp GROUP BY deptid) b
WHERE a.deptid = b.deptid AND a.sal < b.maxsal;

In the example the inline view b returns the details of all department numbers and the maximum salary for each department from emp table. The WHERE a.deptid = b.deptid AND a.sal < b.maxsal clause of the main query displays employee names, salaries, department numbers, and maximum salaries for all the employees who earn less than the maximum salary in their department.

- **What is Wrong with this statement ?**

SELECT empid, name FROM emp
WHERE sal =(SELECT min(sal) FROM emp GROUP BY deptid);

In the SQL statement above, the sub-query contains a GROUP BY clause, which implies that the sub-query will return multiple rows, one for each group it finds. The outer query takes the results of the sub-query and uses these results in its WHERE clause. The

WHERE clause contains an equal (=) operator, a single row comparison operator only one value. The = operator cannot accept more than one value from the sub-query and therefore generates the error. To correct the error, change the = operator to IN.

- **Multiple Row sub-queries :-**

Sub-queries that return more than one row are called multiple row sub-queries. You use a multiple row operator instead of single row operator, with a multiple – row sub-query. The multiple row operator expects one or more values.

```
SELECT empid, name FROM emp
WHERE sal IN (SELECT min(sal) FROM emp GROUP BY deptid);
```

IN :- Equals to any number in the list
 ANY :- Compare value to each value returned by the sub-query.
 ALL :- Compare value to every value returned by the sub-query.

The NOT operator can be used with all above multiple row operators.

```
SELECT empid, name, design, sal FROM emp
WHERE sal < ANY (SELECT sal FROM emp WHERE design = 'PROG')
AND design <> 'PROG';
```

The ANY operator compares a value to each value returned by a sub-query. The above example displays employees who are not programmers and whose salary is less than that of any programmer.

< ANY means less than the maximum. > ANY means more than the minimum
 = ANY is equivalent to IN.

< ALL means less than the minimum. > ALL means more than the Maximum.

```
SELECT empid, name, design, sal FROM emp
WHERE sal < ALL (SELECT sal FROM emp WHERE design = 'PROG')
AND design <> 'PROG';
```

- **NULL values in a Sub-query :-**

```
SELECT emp.name FROM emp
WHERE emp.empid NOT IN (SELECT mgr.mgrno FROM emp mgr);
```

The SQL statement above attempts to display all the employees who do not have any subordinates. Logically this SQL statement should have returned some rows. However, the SQL statement does not return any rows. One of the value returned by the inner query is a null value and hence the entire query returns no rows. The reason is that all conditions that compare a null value result in a null. So whenever null values are likely to be part of the results set of a sub-query, So do not use the NOT IN operator. To correct this use IN operator otherwise write following query.

```
SELECT emp.name FROM emp
WHERE emp.empid NOT IN (SELECT mgr.mgrno FROM emp mgr WHERE mgr.mgrno
IS NOT NULL);
```

- **Multi Column sub-query** :-

Find out all the customers having same names as the employees name.

```
SELECT fname, lname FROM cust_mstr
WHERE (fname, lname) IN (SELECT fname, lname FROM emp_mstr);
```

- Sub-Query in an ORDER BY clause

```
SELECT empno, name, deptno FROM emp ORDER BY (SELECT dname FROM dept);
```

- **INDEXES** :- An oracle server index is an object that can speed up the retrieval of rows by using a pointer. Indexes can be created explicitly or automatically. If you do not have an index on the column, then a full table scan occurs.

An index provide direct and fast access to rows in a table. Its purpose is to reduce the necessity of disk I/O by using an indexed path to locate data quickly. The index is used and maintained automatically by the Oracle Server. Once an index is created. No direct activity is required by the user. When you drop table corresponding indexes are also dropped.

- **Automatically** :- A unique index is created is created automatically by oracle server when you define a column in a table to have a PRIMARY KEY or a UNIQUE key constraint. The name of the index is the name given to the constraint.

- **Manually** :- The other type of index is Non unique index, which a user can create on column to speed up access to the rows.

- **Creating an Index** :- An index created on a single column of a table is called a Simple Index that allows duplicate values.

SYNTAX : CREATE INDEX <index name> ON <table name> (<column name>);

```
CREATE INDEX emp_name_idx ON employees (last_name);
```

An index create on more than one column is called a composite index. The syntax is

```
CREATE INDEX <index name> ON <table name> (<column name1>,<column name2>);
```

```
CREATE INDEX emp_no_mgrno_idx ON <employees> (empno, mgrno);
```

A unique index can also be created on one or more columns. If an index is created on a single column, It is called a Simple Unique Index. If an index is created on more than one column, It is called a Composite Unique Index. The syntax is

```
CREATE UNIQUE INDEX <index name> ON <table name> (<column name>);
```

```
CREATE UNIQUE INDEX <index name> ON <table name> (<column name1>,<column name2>);
```

- **Reverse Key Index** :- In reverse key index, reverses each byte of the column being indexed while keeping the column order. Such an arrangement can help avoid performance degradation in indexes where modifications to the index are concentrated on a small set of blocks. By reversing the keys of the index, the insertions become distributed all over the index.

The column value is stored in an index as follows.

In normal Index	In reverse Index
T1	1T
T2	2T
T3	3T

Under some circumstances, using a reverse key index can make an application run faster.

CREATE INDEX <index name> ON <table name> (<column name>) REVERSE;

A reverse key index can be rebuilt into a normal index using the keywords, REBUILD NONREVERSE.

ALTER INDEX <index name> REBUILD NONREVERSE;

○ **Bitmap Indexes** :-

The advantages of using bitmap indexes are greatest for columns in which the number of distinct values is small compared to the number of rows in the table. If the values in a column are repeated more than a hundred times, the column is candidate for a bitmap index. The syntax is :

CREATE BITMAP INDEX <index name> ON <table name> (<column name>);

Bitmap index provide the following benefits :

- Reduced response time for large classes of ad hoc queries.
- A substantial reduction of space usage compared to other indexing technique.
- Dramatic performance gains even on very low end hardware.

○ **Function based Index** :-

A column's index will not be used when the same column is expressed in an arithmetic expression or function in the WHERE clause. To facilitate such an operation Oracle allows creating indexes based on a function or expression mapped to one or more columns in a table. Function based indexes are very useful when the where clause contains functions or expressions to evaluate a query. This index cannot be created on LOB column, REF column, nested table column. The syntax is

CREATE INDEX <index name> ON <table name> (function name(<column name>));

CREATE INDEX emp_U_name_idx ON employees (upper (last_name));

○ **Multiple Indexes** :-

The Oracle engine allows creation of multiple indexes on each table. The Oracle engine prepares a query plan to decide on the index that must be used for specific data retrieval based on the WHERE clause or the ORDER BY clause specified in the SELECT statement.

- If the SELECT statement is fired without a where clause and Order by clause the Oracle engine does not use the indexes created on the table for data extraction.
- If a WHERE clause or an ORDER BY clause is specified, the oracle engine uses the index created on a column on which the where clause or the order by clause is specified.

- If there is no index for the column specified in the WHERE clause or the ORDER BY clause is not created, data is retrieved without using index.

- **When the oracle engine uses an Index :-**

A SELECT statement with WHERE or ORDER BY clause specified on the column on which an index exist.

- **When the oracle engine does not use an Index :-**

- 1) A SELECT statement without search criteria and order by clause.
- 2) A SELECT statement with WHERE or ORDER BY clause specified on the column on which an index exist.

- More index on a table does not mean faster queries. Each DML operation that is committed on a table with indexes means that the indexes must be updated. The more indexes you have associated with a table, the more efforts the Oracle server must make to update all the indexes after every DML statement.

- **When to create an Index :-**

- 1) The column contains a wide range of values
- 2) The column contains a large number of null values
- 3) One or more columns are frequently used together in a WHERE clause or join condition
- 4) The table is large and most queries are expected to retrieve less than 2 – 4 % of the rows.

- **When not to create an Index :-**

- 1) The table is small
- 2) The columns are not often used as a condition in the query
- 3) Most queries are expected to retrieve more than 2 to 4 percent of the rows in the table.
- 4) The table is updated frequently
- 5) The indexed columns are referenced as part of an expression.

- Confirming index by uses USER_INDEXES and USER_IND_COLUMNS datadictionary

- **Removing an Index :**

You cannot modify indexes. To change an index, you must drop it and then re-create it. Remove an index definition from data dictionary by issuing the DROP INDEX statement. To drop an index, you must be an owner of the index or have the DROP ANY INDEX privilege.

The syntax is : DROP INDEX <index name>;

- **SEQUENCES :-**

A Sequence is a user created database object that can be shared by multiple users to generate unique integers. A typical usage for sequences is to create a primary key value, which must be unique for each row. The sequence is generated and incremented by an internal Oracle routine. This can be a time saving object because it can reduce the amount of application code needed to write a sequence-generating routine.

Sequence numbers are stored and generated independently of tables. Therefore, the same sequence can be used for multiple tables. The value generated by sequence can have a maximum of 38 digits. A sequence can be defined to:

- Generate numbers in ascending or descending order.
- Provide interval between numbers.
- Caching of sequence number in memory to speed up their availability.

- **Creating Sequence :-**

The minimum information required for generating numbers using a sequence is :

- 1) The starting number
- 2) The maximum number that can be generated by a sequence.
- 3) The increment value for generating the next number.

The Syntax is :

```
CREATE SEQUENCE <sequence name>
[ INCREMENT BY <integer value>
  START WITH <integer value>
  MAX VALUE <integer value> / NOMAXVALUE
  MINVALUE <integer value> / NOMINVALUE
  CYCLE / NOCYCLE
  CACHE <integer value> / NOCACHE
  ORDER / NOORDER ]
```

INCREMENT BY :- Specifies the interval between sequence numbers. It can be any positive or negative value but not zero. If this value is omitted, the default value is 1.

MAXVALUE :- Specifies the sequence maximum value

NOMAXVALUE :- Specifies a maximum value of 10^{27} for an ascending sequence and -1 for a descending sequence. This is the default clause.

MINVALUE :- Specifies the sequence minimum value

NOMINVALUE :- Specifies a minimum value of 1 for an ascending sequence and -10^{26} for a descending sequence.

START WITH :- Specifies the first sequence number to be generated. The default for an ascending sequence minimum value 1 and for a descending sequence, it is the maximum value -1.

CYCLE :- Specifies that the sequence continues to generate repeat values after reaching either its maximum value.

NOCYCLE :- Specifies that a sequence cannot generate more values after reaching the maximum value.

CACHE :- Specifies how many values of a sequence Oracle pre allocates and keeps in memory for faster access. The minimum value for this parameter is two.

NOCACHE :- Specifies that values of a sequence are not pre-allocated.

```
CREATE SEQUENCE dept_deptid_seq INCREMENT BY 10
START WITH 50
MAXVALUE 9999 NOCACHE NOCYCLE ;
```

- **Confirming Sequence :-** Verify your sequence values in the USER_SEQUENCES data dictionary table.

- **NEXTVAL and CURRVAL :-** After you creating your sequence, it generates sequential numbers for use in your tables. Reference the sequence values by using the NEXTVAL and CURRVAL pseudocolumns.

The NEXTVAL is used to extract successive sequence numbers from a specified sequence. You must qualify NEXTVAL with the sequence name. When you reference sequence.NEXTVAL, a new sequence number is generated and the current sequence number is placed in CURRVAL.

The CURRVAL is used to refer to a sequence number that the current user has just generated. NEXTVAL must be used to generate a sequence number in the current user's session before CURRVAL can be referenced. You must qualify CURRVAL with the sequence name. When sequence.CURRVAL is referenced, the last value returned to that user's process is displayed.

You can use NEXTVAL and CURRVAL in the following contexts:

- 1) The SELECT list of a SELECT statement that is not a part of a subquery.
- 2) The SELECT list of a sub-query in an INSERT statement.
- 3) The VALUES clause of an INSERT statement.
- 4) The SET clause of an UPDATE statement.

You cannot use NEXTVAL and CURRVAL in the following context

- 1) The SELECT list of view
- 2) A SELECT statement with the DISTINCT keyword
- 3) A SELECT statement with GROUP BY, HAVING, or ORDER BY clauses.
- 4) A sub-query in a SELECT, DELETE, or UPDATE statement
- 5) The DEFAULT expression in a CREATE TABLE or ALTER TABLE statement.

```
INSERT INTO departments (department_id, department_name, location_id)
VALUES (dept_deptid_seq.NEXTVAL, 'SUPPORT' 2500);
```

```
SELECT dept_deptid_seq.CURRVAL FROM dual;
```

○ **Modifying Sequence :-**

If you reach the MAXVALUE limit for your sequence, no additional values from the sequence are allocated and you will receive an error indicating that the the sequence exceeds the MAXVALUE. To continue to use the sequence, you can modify it by using the ALTER SEQUENCE statement.

```
ALTER SEQUENCE <sequence name>
[ INCREMENT BY <integer value>
  START WITH <integer value>
  MAX VALUE <integer value> / NOMAXVALUE
  MINVALUE <integer value> / NOMINVALUE
  CYCLE / NOCYCLE
  CACHE <integer value> / NOCACHE
  ORDER / NOORDER ]
```

Guide line for modifying sequences

- 1) You must be the owner or have the ALTER privilege for the sequence in order to modify it
- 2) Only future sequence numbers are affected by the ALTER SEQUENCE statement.

- 3) The START WITH option cannot be changed using ALTER SEQUENCE. The sequence must be dropped and re-created in order to restart the sequence at a different number.
- 4) Some validation is performed. F.e. a new MAXVALUE that is less than the current sequence number cannot be imposed.

○ **Removing a Sequence** :-

To remove a sequence from the data dictionary, use the DROP SEQUENCE statement. You must be the owner of the sequence or have the DROP ANY SEQUENCE privilege to remove it. Syntax is : DROP SEQUENCE <sequence name>.

• **VIEWS** :-

○ **What is a View?** :-

You can present logical subsets or combinations of data by creating views of tables. A view is a logical table based on a table or another view. A view contains no data of its own but is like a window through which data from tables can be viewed or changed. The tables on which a view is based are called base tables. The view is stored as a SELECT statement in the data dictionary.

If a view is used to only look at table data and nothing else the view is called **read only view**. A view that is used to look at table data as well as Insert, Update, and Delete table data is called an Updateable view.

Mainly because of two reasons you can create a view. First is When data security is required and Second is When data redundancy is to be kept to the minimum while maintaining data security.

○ **Advantages of views :**

- 1) Views restrict access to the data because the view can display selective columns from the table.
- 2) Views can be used to make simple queries to retrieve the results of complicated queries. F.e. views can be used to query information from multiple tables without the user knowing how to write join statement.
- 3) Views provide data independence for ad hoc users and application programs. One view can be used to retrieve data from several tables.
- 4) Views provide groups of users access to data according to their particular criteria.

Syntax is :

```
CREATE VIEW <view name> AS SELECT <col1>, <col2> FROM <table name>
WHERE <column name> = <expression list> GROUP BY <grouping criteria>
HAVING <predicate>
```

```
CREATE VIEW empvu30 AS SELECT empno, name, sal FROM emp
WHERE deptno = 30;
```

```
DESC empvu30;
```

```
CREATE VIEW salvu20 AS SELECT empno id_no, name emp_name, sal * 12 "Ann
Sal" FROM emp WHERE deptno = 20;
```

Once a view has been created, it can be queried exactly like a base table.

```
SELECT <col1>, <col2> FROM <view name>;
```

- **Updateable Views :**

Views can also be used for data manipulation views on which data manipulation can be done are called Updateable views. When an updateable view name is given in an insert Update, or Delete SQL statement, modifications to data in the view will be immediately passed to the underlying table.

For a view to be updateable, it should meet the following criteria :

- 1) Views defined from one table.
- 2) If the user wants to INSERT records with the help of a view, then the PRIMARY KEY columns and all the NOT NULL columns must be included in the view.
- 3) The user can UPDATE , DELETE records with the help of a view even if the PRIMARY KEY column and NOTNULL columns are excluded from the view definition.

```
CREATE VIEW empvu AS SELECT empno, name, sal FROM emp;
```

```
INSERT INTO empvu VALUES (7777, 'Ashish', 12000);
```

```
UPDATE empvu SET name = 'Tanuj' WHERE name = 'Ashish';
```

```
DELETE FROM empvu WHERE name = 'Tanuj';
```

If a view is created from multiple tables, which were not created using a Referencing clause, then though the PRIMARY KEY columns as well as the NOT NULL columns are included in the view definition the view's behavior will be The INSERT, UPDATE, or DELETE operations is not allowed

If a view is created from multiple tables, which were created using a Referencing clause, then though the PRIMARY KEY columns as well as the NOT NULL columns are included in the view definition the view's behavior will be

- 1) An INSERT operations is not allowed
- 2) The DELETE or MODIFY operations do not affect the Master table.
- 3) The view can be used to MODIFY the columns of the detail table included in the view.

If a DELETE operation is executed on the view, the corresponding records from the detail table will be deleted.

```
CREATE emp_dept_vu ( empname, Deptname, salary) AS SELECT e.name,d.dname,e.sal  
FROM emp e, dept d WHERE e.deptno=d.deptno;
```

Common restrictions on Updateable Views :- The following condition holds true irrespective of the view being created from a single table or multiple tables.

For the view to be updateable the view definition must not include :

- 1) Aggregate functions
- 2) DISTINCT, GROUP BY or HAVING clause
- 3) Sub-queries.
- 4) Constants, Strings or Value Expressions like Sell_price * 1.05
- 5) UNION, INTERSECT or MINUS clause.
- 6) If a view is defined from another view, the second view should be updateable..

If the user tries to perform any of INSERT, UPDATE, DELETE operation, on a view, which is created from a non updateable view Oracle returns the error message “ Data manipulation operation not legal on this view”

The DROP VIEW command is used to remove a view from the database. Syntax is :
DROP VIEW <view name>

- **SYNONYMS :-**

To refer to a table owned by another user, you need to prefix the table name with the name of the user who created it followed by a period (.). Creating a synonym eliminates the need to qualify the object name with the schema and provides you with an alternative name for a table, view, sequence, procedure, or other objects. This method can be especially useful with lengthy object names, Such as views.

Syntax :- CREATE [Public] SYNONYMS <synonym name> FOR <object name>

PUBLIC	:-	Creates a synonym accessible to all users.
Synonym name	:-	Is the name of the synonym to be created.
Object name	:-	Identifies the object for which the synonym is created.

Guideline :-

- 1) The object cannot be contained in a package.
- 2) A private synonym name must be distinct from all other objects owned by the same user.
- 3) The database administrator or the object owner can create public synonym accessible to all users.

CREATE SYNONYM d_sum FOR scott.dept_sum_vu ;

CREATE PUBLIC SYNONYM dept FOR nvp.departments;

To remove synonym, use the DROP SYNONYM statement. Only the database administrator can drop a public synonym.

DROP PUBLIC SYNONYM dept ;

→ **TABLE ALIAS :-**

Qualifying column names with table names can be very time consuming, particularly if table names are lengthy. You can use table aliases instead of table names. Just as a column alias gives a column another name, a table alias gives a table another name. Table aliases help to keep SQL code smaller, therefore using less memory.

Guide line :-

- Table aliases can be up to 30 character in length, but shorter is better.
- If a table alias is used for a particular table name in the FROM clause, then that table alias must be substituted for the table name throughout the SELECT statement.
- Table aliases should be meaning full.
- The table alias is valid only for the current SELECT statement.

JOIN :- SQL Joins are used to relate information in different tables. A Join condition is a part of the sql query that retrieves rows from two or more tables. A SQL Join condition is used in the SQL WHERE Clause of select, update, delete statements.

Oracle / Theta Style

Equi Join
Outer join
Self Join
Non – Equi Join
Cartesian Product

SQL : 1999 / Ansi Style

Natural / Inner Join
Left / Right Outer Join
Join On
Join Using
Cross Join

The Syntax for joining two tables is:

```
SELECT col1, col2, col3...
```

```
FROM table_name1, table_name2
```

```
WHERE table_name1.col2 = table_name2.col1;
```

If a sql join condition is omitted or if it is invalid the join operation will result in a Cartesian product. The Cartesian product returns a number of rows equal to the product of all rows in all the tables being joined. For example, if the first table has 20 rows and the second table has 10 rows, the result will be $20 * 10$, or 200 rows. This query takes a long time to execute.

Equi join : All the rows returned by the sql query satisfy the sql join condition specified.
database table "product";

product_id	product_name	supplier_name	unit_price
100	Camera	Nikon	300
101	Television	Onida	100
102	Refrigerator	Vediocon	150
103	Ipod	Apple	75
104	Mobile	Nokia	50

database table "order_items";

order_id	product_id	total_units	customer
5100	104	30	Infosys
5101	102	5	Satyam
5102	103	25	Wipro
5103	101	10	TCS

For example: If you want to display the product information for each order the query will be as given below. Since you are retrieving the data from two tables, you need to identify the common column between these two tables, which is the product_id.
The query for this type of sql joins would be like,

```
SELECT order_id, product_name, unit_price, supplier_name, total_units
```

```
FROM product, order_items
```

```
WHERE order_items.product_id = product.product_id;
```

The columns must be referenced by the table name in the join condition, because product_id is a column in both the tables and needs a way to be identified. This avoids ambiguity in using the columns in the SQL SELECT statement.

The number of join conditions is (n-1), if there are more than two tables joined in a query where 'n' is the number of tables involved. The rule must be true to avoid Cartesian product.

We can also use aliases to reference the column name, then the above query would be like,

```
SELECT o.order_id, p.product_name, p.unit_price, p.supplier_name, o.total_units
```

```
FROM product p, order_items o
```

```
WHERE o.product_id = p.product_id;
```

→ **NON EQUI JOIN :-**

A non-equijoin is a join condition containing something other than an equality operator.

```
SELECT e.name, e.sal, j.grade FROM emp e, salgrade s WHERE e.sal between s.lowsal and s.highsal
```

The example creates a non-equijoin to evaluate an employee's salary grade. The salary must be between any pair of the low and high salary ranges.

It is important to note that all employees appear exactly once when this query is executed. No employee is repeated in the list. There are two reasons for this:

- None of the rows in the job grade table contain grades that overlap. That is, the salary value for an employee can lie only between the low salary and high salary values of one of the rows in the salgrade table.
- All of the employees salaries lie within the limits provided by the salgrade table. That is, no employees earns less than the lowest value contained in the lowsal column or more than the highest value contained in the Highsal column.

Note :- Other conditions, such as <= and >= can be used, but BETWEEN is the simplest. Remember to specify the low value first and the high value last when using BETWEEN.

→ **OUTER JOIN :-**

It returns a record with no direct match. It can be used when a row does not satisfy a join condition, the row will not appear in the query result.

The missing rows can be returned if an outer join operator is used in the join condition. The operator is a plus sign enclosed in parentheses (+), and it is placed on the "side" of the join that is

deficient in information. This operator has the effect of creating one or more null rows, to which one or more rows from the nondeficient table can be joined.

SELECT t1.col, t2.col FROM t1,t2 WHERE t1.col(+) = t2.col; OR

SELECT t1.col, t2.col FROM t1 LEFT OUTER JOIN t2 ON (t2.col = t1.col)

SELECT t1.col, t2.col FROM t1,t2 WHERE t1.col = t2.col (+); OR

SELECT t1.col, t2.col FROM t1 RIGHT OUTER JOIN t2 ON (t2.col = t1.col)

T1.col = → is the condition that joins or relates the tables together.

T2.col(+)→ is the outer join symbol, which can be placed on either side of the WHERE clause condition, but not on both sides. Place the outer join symbol following the name of the column in the table without the matching rows.

SELECT e.name, e.deptid, d.dname FROM emp e, dept d WHERE d.deptid (+) = e.deptid;

OR

SELECT e.name, e.deptid, d.dname FROM emp e LEFT OUTER JOIN dept d ON e.deptid = d.deptid;

This query retrieves all rows in the EMP table, which is the left table even if there is no match in the DEPT table.

SELECT e.name, e.deptid, d.dname FROM emp e, dept d WHERE d.deptid = e.deptid (+);

OR

SELECT e.name, e.deptid, d.dname FROM emp e RIGHT OUTER JOIN dept d ON e.deptid = d.deptid;

This query retrieves all rows in the DEPT table, which is the right table even if there is no match in the EMP table.

SELECT e.name, e.deptid, d.dname FROM emp e FULL OUTER JOIN dept d ON e.deptid = d.deptid

This query retrieves all rows in the EMP table, even if there is no match in the DEPT table. It also retrieves all rows in the DEPT table even if there is no match in the EMP table.

Outer join Restriction :-

- The outer join operator can appear on only one side of the expression – the side that has information missing. It returns those rows from one table that have no direct match in the other table.
- A condition involving an outer join cannot use the IN operator or be linked to another condition by the OR operators.

→ **SELF JOIN :-**

Some times you need to join a table itself. To find the name of each employee's manager, you need to join the employee table to itself or perform a self join.

EMPLOYEES (Worker)

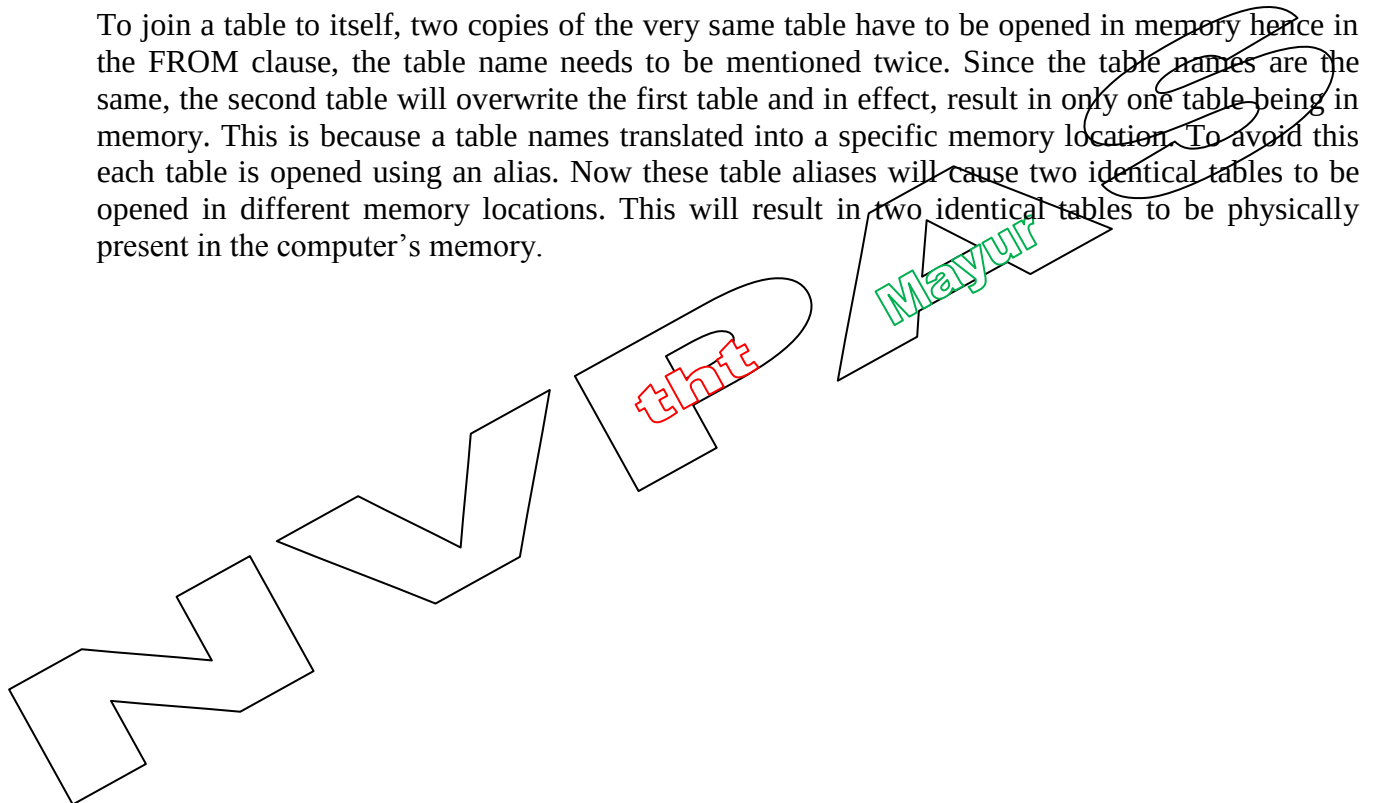
Empid
Lname
Mgr

EMPLOYEES (Manager)

Empid
Lname

```
SELECT E.NAME, E.EMPID, M.NAME, M.EMPID FROM EMP E, EMP M WHERE E.MGR=
M.EMPID
```

To join a table to itself, two copies of the very same table have to be opened in memory hence in the FROM clause, the table name needs to be mentioned twice. Since the table names are the same, the second table will overwrite the first table and in effect, result in only one table being in memory. This is because a table names translated into a specific memory location. To avoid this each table is opened using an alias. Now these table aliases will cause two identical tables to be opened in different memory locations. This will result in two identical tables to be physically present in the computer's memory.



TOPICS

Transaction Management and Reporting through SQL*Plus

- Data control language statements – GRANT and REVOKE
- Transaction control language statements – COMMIT, ROLLBACK and SAVEPOINT
- Overview of SQL*Plus report
- Reporting commands – remark, setheadsep, ttitle, btitle, column, break on, compute, spool, set pause.

Data Control Language :

- **SECURITY** :- Objects that are created by a user are owned and controlled by that user. If a user wishes to access any of the objects belonging to another user, the owner of the object will have to give permissions for such access. This is called **Granting of Privileges**. Privileges once given can be taken back by the owner of the object. This is called **Revoking of Privileges**.

- **GRANTING PRIVILEGES** :-

Syntax :-

GRANT <object privileges> [<columns>] / All

ON <object name>

TO {user / Role/ PUBLIC}

[WITH GRANT OPTION];

Object Privileges : Each object privilege that is granted authorizes the grantee to perform some operation on the objects. A user can grant all the privileges or grant only specific object privileges.

The list of object privileges are :

ALTER : Allows the grantee to change the table definition with the ALTER TABLE command.

DELETE : Allows the grantee to remove the records from the table with the DELETE command.

INDEX : Allows the grantee to create an index on the table with the CREATE INDEX command.

INSERT : Allows the grantee to add records to the table with the INSERT command.

SELECT : Allows the grantee to query the table with the SELECT command.

UPDATE : Allow the grantee to modify the records in the tables with the UPDATE command.

ALL : specifies all object privileges

Columns : specifies the column from a table or view on which privileges are granted.

ON object : is the object on which the privileges is granted

TO : identifies to whom the privileges is granted.

PUBLIC : grants object privileges to all users.

WITH GRANT : allows the grantee to grant the object privileges to other users.

OPTION

Guideline :

- 1) To grant privileges on an object, the object must be in your own schema, or you must have been granted the object privileges WITH GRANT OPTION.
- 2) An object owner can grant any object privilege on the object to any other user or role of the database.

3) The owner of an object automatically acquires all object privileges on that object.

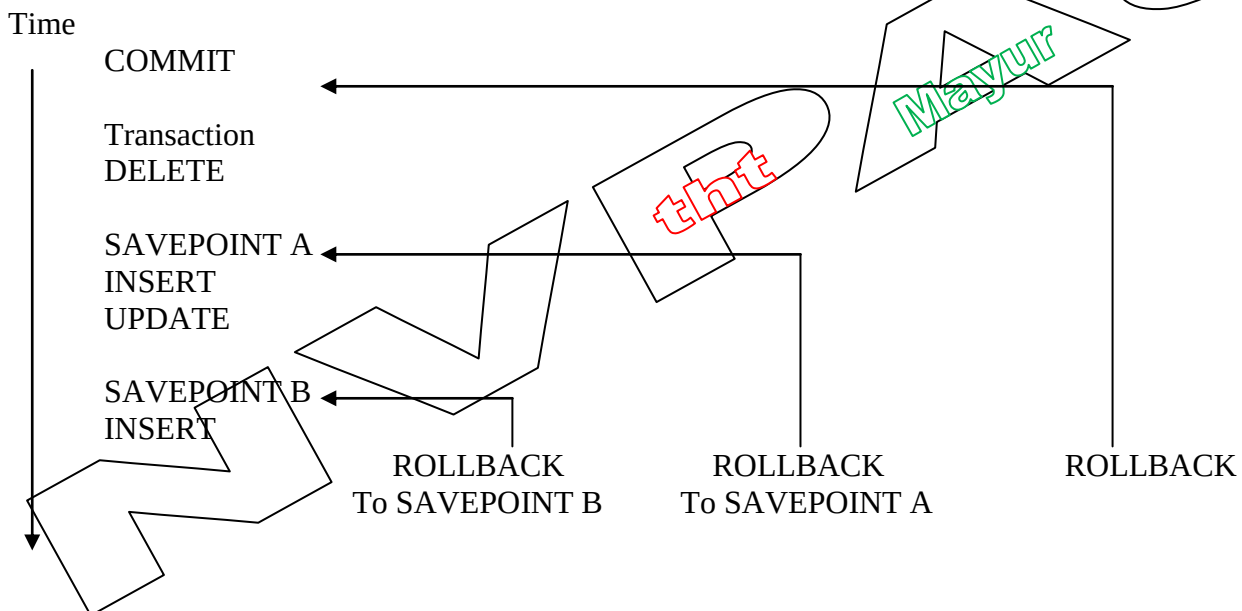
Ex1 : GRANT select ON emp TO sybca631, sybca632;
 Ex2 : GRANT insert, update(deptname) ON dept TO sybca631, sybca632;
 Ex3 : GRANT ALL ON dept TO scott WITH GRANT OPTION ;
 Ex4 : after connecting as scott user
 GRANT select ON sybca.dept TO PUBLIC;

- **REVOKING PRIVILEGES** :- Privileges once given can be denied to a user using the REVOKE command. The object owner can revoke privileges granted to another user. A user of an object who is not the owner, but has been granted the GRANT privilege, has the power to REVOKE the privileges from a grantee.

Syntax :

REVOKE < object privileges>
 ON < objectname>
 FROM <username>
 REVOKE ALL ON dept FROM scott ;

Transaction Control Language:



- **COMMIT** :- A commit ends the current transaction and makes permanent any changes made during the transaction. All transactional locks acquired on tables are released.
 Syntax :- COMMIT
- **ROLLBACK** :- A ROLLBACK does exactly the opposite of COMMIT. It ends the transaction but undoes any changes made during the transaction. All transactional locks acquired on tables are released.
 Syntax :- ROLLBACK / ROLLBACK TO SAVEPOINT <savepoint name>

SAVEPOINT :- Is optional and is used to rollback a transaction partially, as far as the specified savepoint.

SAVEPOINT NAME :- Is a savepoint created during the current transaction.

ROLLBACK can be fired from the SQL prompt with or without the SAVEPOINT clause.

A ROLLBACK operation performed without the SAVEPOINT clause amounts to the following :

- 1) Ends the transaction
- 2) Undoes all the changes in the current transaction.
- 3) Erase all savepoints in that transaction.
- 4) Release the transactional locks.

A ROLLBACK operation performed with the TO SAVEPOINT clause amounts to the following :

- 1) A predetermined portion of the transaction is rolled back
- 2) Retains the save point rolled back to , but loses those created after the named savepoint.
- 3) Release all transactional locks that were acquired since the savepoint was taken.

- Creating SAVEPOINT :- SAVEPOINT marks and saves the current point in the processing of a transaction. When a SAVEPOINT is used with a ROLLBACK statement, parts of a transaction can be undone. An active SAVEPOINT is one that specified since the last COMMIT or ROLLBACK.

Syntax :- SAVEPOINT <savepoint name>

→ SQL REPORTS :-

Formatting Columns

Through the SQL*Plus COLUMN command, you can change the column headings and reformat the column data in your query results.

Changing Column Headings

When displaying column headings, you can either use the default heading or you can change it using the COLUMN command. The following sections describe how default headings are derived and how to alter them using the COLUMN command.

You can define a more useful column heading with the HEADING clause of the COLUMN command, in the following format:

COLUMN column_name HEADING column_heading

Example :- Changing a Column Heading

To produce a report from EMP_DETAILS_VIEW with new headings specified for LAST_NAME, SALARY, and COMMISSION_PCT, enter the following commands:

```
COLUMN LAST_NAME      HEADING 'LAST NAME'
COLUMN SALARY          HEADING 'MONTHLY SALARY'
COLUMN COMMISSION_PCT HEADING 'COMMISSION'
SELECT LAST_NAME, SALARY, COMMISSION_PCT
FROM EMP_DETAILS_VIEW
WHERE JOB_ID='SA_MAN';
```

LAST NAME	MONTHLY SALARY	COMMISSION
Russell	14000	.4
Partners	13500	.3
Errazuriz	12000	.3
Cambrault	11000	.3
Zlotkey	10500	.2

Note:

The new headings will remain in effect until you enter different headings, reset each column's format, or exit from SQL*Plus.

To change a column heading to two or more words, enclose the new heading in single or double quotation marks when you enter the COLUMN command. To display a column heading on more than one line, use a vertical bar (|) where you want to begin a new line.

Example :- Splitting a Column Heading

To give the columns SALARY and LAST_NAME the headings MONTHLY SALARY and LAST NAME respectively, and to split the new headings onto two lines, enter

```
COLUMN SALARY HEADING 'MONTHLY|SALARY'
```

```
COLUMN LAST_NAME HEADING 'LAST|NAME'
```

Now rerun the query with the slash (/) command:

/

LAST NAME	MONTHLY SALARY	COMMISSION
Russell	14000	4
Partners	13500	3
Errazuriz	12000	3
Cambrault	11000	3
Zlotkey	10500	2

The COLUMN command identifies the column you want to format and the model you want to use, as shown:

```
COLUMN column_name FORMAT model
```

Use format models to add commas, dollar signs, angle brackets (around negative values), and leading zeros to numbers in a given column. To use more than one format model for a single column, combine the desired models in one COLUMN command.

Example :- Formatting a NUMBER Column

To display SALARY with a dollar sign, a comma, and the numeral zero instead of a blank for any zero values, enter the following command:

```
COLUMN SALARY FORMAT '$99,990'
```

Now rerun the current query:

/

LAST NAME	MONTHLY SALARY	COMMISSION
Russell	\$14,000	.4
Partners	\$13,500	.3
Errazuriz	\$12,000	.3
Cambrault	\$11,000	.3
Zlotkey	\$10,500	.2

Use a zero in your format model, as shown, when you use other formats such as a dollar sign and wish to display a zero in place of a blank for zero values.

Example :- Formatting a Character Column

To set the width of the column LAST_NAME to four characters and rerun the current query, enter
 COLUMN LAST_NAME FORMAT A4
 /

LAST NAME	MONTHLY SALARY	COMMISSION
-----	-----	-----
Russ ell	\$14,000	.4
Part ners	\$13,500	.3
Erra	\$12,000	.3

If the WRAP variable of the SET command is set to ON (its default value), the employee names wrap to the next line after the fourth character, as shown in Example_. If WRAP is set to OFF, the names are truncated (cut off) after the fourth character.

Note:

The column heading is truncated regardless of the setting of WRAP or any COLUMN command clauses.

Now return the column to its previous format:

COLUMN LAST_NAME FORMAT A10

To reset the display attributes for a column to their default values, use the CLEAR clause of the COLUMN command as shown:

COLUMN column_name CLEAR

Example :- Resetting Column Display Attributes to their Defaults

To reset all column display attributes to their default values, enter:

CLEAR COLUMNS

columns cleared

Suppressing Duplicate Values in Break Columns

The BREAK command suppresses duplicate values by default in the column or expression you name. Thus, to suppress the duplicate values in a column specified in an ORDER BY clause, use the BREAK command in its simplest form:

BREAK ON break_column

Note:

Whenever you specify a column or expression in a BREAK command, use an ORDER BY clause specifying the same column or expression. If you do not do this, breaks occur every time the column value changes.

Example :- Suppressing Duplicate Values in a Break Column

To suppress the display of duplicate department numbers in the query results shown, enter the following commands:

BREAK ON DEPARTMENT_ID;

For the following query (which is the current query stored in the buffer):

```
SELECT DEPARTMENT_ID, LAST_NAME, SALARY
FROM EMP_DETAILS_VIEW
WHERE SALARY > 12000
ORDER BY DEPARTMENT_ID;
```


DEPARTMENT_ID	LAST_NAME	SALARY
20	Hartstein	13000
80	Russell	14000
	Partners	13500
90	King	24000
	Kochhar	17000
	De Haan	17000

6 rows selected.

Inserting Space when a Break Column's Value Changes

You can insert blank lines or begin a new page each time the value changes in the break column. To insert *n* blank lines, use the BREAK command in the following form:

`BREAK ON break_column SKIP n`

To skip a page, use the command in this form:

`BREAK ON break_column SKIP PAGE`

Example :- Inserting Space when a Break Column's Value Changes

To place one blank line between departments, enter the following command:

`BREAK ON DEPARTMENT_ID SKIP 1`

Now rerun the query:

DEPARTMENT_ID	LAST_NAME	SALARY
20	Hartstein	13000
80	Russell	14000
	Partners	13500
90	King	24000
	Kochhar	17000
	De Haan	17000

6 rows selected.

Listing and Removing Break Definitions

You can remove the current break definition by entering the CLEAR command with the BREAKS clause:

`CLEAR BREAKS`

You may wish to place the command CLEAR BREAKS at the beginning of every script to ensure that previously entered BREAK commands will not affect queries you run in a given file.

Computing Summary Lines when a Break Column's Value Changes

If you organize the rows of a report into subsets with the BREAK command, you can perform various computations on the rows in each subset. You do this with the functions of the SQL*Plus COMPUTE command. Use the BREAK and COMPUTE commands together in the following forms:

`BREAK ON break_column`

`COMPUTE function LABEL label_name OF column column column`

`... ON break_column`

You can include multiple break columns and actions, such as skipping lines in the BREAK command, as long as the column you name after ON in the COMPUTE command also appears after ON in the BREAK command. To include multiple break columns and actions in BREAK when using it in conjunction with COMPUTE, use these commands in the following forms:

`BREAK ON break_column_1 SKIP PAGE ON break_column_2 SKIP 1`

`COMPUTE function LABEL label_name OF column column column`

`.. ON break_column_2`

The COMPUTE command has no effect without a corresponding BREAK command.

You can COMPUTE on NUMBER columns and, in certain cases, on all types of columns. The following table lists compute functions and their effects

Table Compute Functions

Function...	Computes the...
SUM	Sum of the values in the column.
MINIMUM	Minimum value in the column.
MAXIMUM	Maximum value in the column.
AVG	Average of the values in the column.
STD	Standard deviation of the values in the column.
VARIANCE	Variance of the values in the column.
COUNT	Number of non-null values in the column.
NUMBER	Number of rows in the column.

The function you specify in the COMPUTE command applies to all columns you enter after OF and before ON. The computed values print on a separate line when the value of the ordered column changes. Labels for ON REPORT and ON ROW computations appear in the first column; otherwise, they appear in the column specified in the ON clause.

You can change the compute label by using COMPUTE LABEL. If you do not define a label for the computed value, SQL*Plus prints the unabbreviated function keyword.

The compute label can be suppressed by using the NOPRINT option of the COLUMN command on the break column.

Example Computing and Printing Subtotals

To compute the total of SALARY by department, first list the current BREAK definition:

BREAK

which displays current BREAK definitions:

```
break on DEPARTMENT_ID page nodup
on JOB_ID skip 1 nodup
```

Now enter the following COMPUTE command and run the current query:

```
COMPUTE SUM OF SALARY ON DEPARTMENT_ID
```

```

DEPARTMENT_ID  JOB_ID  LAST_NAME  SALARY
-----
          20  MK_MAN  Hartstein  13000
*****
sum  13000

DEPARTMENT_ID  JOB_ID  LAST_NAME  SALARY
-----
          80  SA_MAN  Russell    14000
          80  Partners 13500
*****
sum  27500

DEPARTMENT_ID  JOB_ID  LAST_NAME  SALARY
-----
          90  AD_PRES  King       24000
          90  AD_VP   Kochhar    17000
          90  De Haan 17000
*****
sum  58000
```

Example : Removing COMPUTE Definitions

To remove all COMPUTE definitions and the accompanying BREAK definition, enter the following commands:

CLEAR BREAKS

breaks cleared

CLEAR COMPUTES

computes cleared

You may wish to place the commands CLEAR BREAKS and CLEAR COMPUTES at the beginning of every script to ensure that previously entered BREAK and COMPUTE commands will not affect queries you run in a given file.

Defining Page and Report Titles and Dimensions

The word page refers to a screen full of information on your display or a page of a spooled (printed) report. You can place top and bottom titles on each page, set the number of lines per page, and determine the width of each line.

The word report refers to the complete results of a query. You can also place headers and footers on each report and format them in the same way as top and bottom titles on pages.

Setting the Top and Bottom Titles and Headers and Footers

As you have already seen, you can set a title to display at the top of each page of a report. You can also set a title to display at the bottom of each page. The TTITLE command defines the top title; the BTITLE command defines the bottom title.

A TTITLE, BTITLE command consists of the command name followed by one or more clauses specifying a position or format and a CHAR value you wish to place in that position or give that format. You can include multiple sets of clauses and CHAR values:

TTITLE *position_clause(s) char_value position_clause(s) char_value ...*

BTITLE *position_clause(s) char_value position_clause(s) char_value ...*

Example Placing a Top and Bottom Title on a Page

To put titles at the top and bottom of each page of a report, enter

TTITLE CENTER -

"ACME SALES DEPARTMENT PERSONNEL REPORT"

BTITLE CENTER "COMPANY CONFIDENTIAL"

Now run the current query:

ACME SALES DEPARTMENT PERSONNEL REPORT		
DEPARTMENT_ID	LAST_NAME	SALARY
30	Colmenares	2500
30	Himuro	2600
30	Tobias	2800
30	Baida	2900
30	Khoo	3100
30	Raphaely	11000

COMPANY CONFIDENTIAL

6 rows selected.

Complete Example :-

Create a script named EMPRPT containing the following commands.

First, use EDIT to create the script with your operating system text editor.

EDIT EMPRPT

Next, enter the following commands into the file, using your text editor:

SPOOL TEMP

CLEAR COLUMNS

CLEAR BREAKS

CLEAR COMPUTES

COLUMN DEPARTMENT_ID HEADING DEPARTMENT

COLUMN LAST_NAME HEADING 'LAST NAME'

COLUMN SALARY HEADING 'MONTHLY SALARY' FORMAT \$99,999

BREAK ON DEPARTMENT_ID SKIP 1 ON REPORT

COMPUTE SUM OF SALARY ON DEPARTMENT_ID

COMPUTE SUM OF SALARY ON REPORT

SET PAGESIZE 24

SET NEWPAGE 0

SET LINESIZE 70

TTITLE CENTER 'A C M E W I D G E T' SKIP 2 -

LEFT 'EMPLOYEE REPORT' RIGHT 'PAGE:' -

FORMAT 999 SQL.PNO SKIP 2

BTITLE CENTER 'COMPANY CONFIDENTIAL'

SELECT DEPARTMENT_ID, LAST_NAME, SALARY

FROM EMP_DETAILS_VIEW

WHERE SALARY > 12000

ORDER BY DEPARTMENT_ID;

SPOOL OFF

A C M E W I D G E T		
EMPLOYEE REPORT		PAGE: 1
DEPARTMENT	LAST NAME	MONTHLY SALARY
20	Hartstein	\$13,000

	sum	\$13,000
80	Russell	\$14,000
	Partners	\$13,500

	sum	\$27,500
90	King	\$24,000
	Kochhar	\$17,000
	De Haan	\$17,000

	sum	\$58,000
	sum	\$98,500
COMPANY CONFIDENTIAL		
6	rows selected.	

REMARK

This command can be shortened to REM. SQLPLUS ignores anything on a line that begins with REM to allow you to document your program file.

/* */ REMARKS

Use this option to embed comments in an SQL statement. Everything within the symbols is treated as a remark.

SET HEADSEP

This is a Heading Separator command to tell SQLPLUS how you will indicate where you wish to break a page or column heading that is more than one line. The default is the vertical bar (|), but if your keyboard does not have this, use this command to select another character like the exclamation point.

SPOOL

Spooling a file is writing the file to a print queue. To name the output file for your report, use the SPOOL command. We usually use a default filename extension of .lst (stands for listing).

Spool myoutputfile.lst

- Place this command just before your SELECT command in a report producing program.
- Place SPOOL OFF directly after the SELECT command. This turns spooling off so that output quits going to the output file.
- You can also use spooling to create a program file, but all of the output including messages produced by SQLPLUS will go to the file.

SET PAUSE

You can cause the report to display exactly one full screen at a time. I put these commands in my login.sql file so that they automatically execute every time I load SQLPLUS. I use output of 23 lines for the screen.

- I also define the editor to use for my files to be the vi editor. Put all of these commands in your login.sql file.
- set pause 'More ...'
- set pagesize 23
- set pause on